

Company: PhonePe

Job Profile: SDE

Offer Type: Internship + Placement

Internship Stipend: 80K per month

CTC: 33.5L (19.25L Base + 5L Joining bonus + 8.5L RSU)

Location: Pune/ Bangalore

Process Date: 7/7/25: PPT and OA

8/7/25: Interview

Placement Process:

1. **Online Assessment** [219 candidates had applied]

Platform: DoSelect || Questions: 4 || Time: 1hr 30mins

I would advise you to skim through all the questions first and start with the one which seems familiar/easy. I started with Q3, which I solved in 7–8 minutes. Then I moved to Q1, got stuck on an edge case due to limited debugging options, but managed to solve it in 20 minutes. I spent the rest of the time on Q2, implemented a recursive solution, and passed a few test cases.

- 1) You were given n people and 2 array A and B of size m , where each pair $(A[i], B[i])$ means that $A[i]$ is a friend of $B[i]$. Also, friendships are transitive, so if x is a friend of y and y is friend of z , x is also friend of z . You were told to find the **minimum no of new friendship** such that everyone is friend is each other.

Approach: The question can be easily solved by counting the no of components. I used **DSU** to find the **no of connected components** and return it -1, A DFS or BFS approach would have worked too.

- 2) You were given an array A which represented the laddoos given to n employees. Each person must receive atleast 1 and atmost m laddoos. You had to assign laddoos to employed who had received 0 laddoos such that the absolute difference between two employees is not more than k . And finally, you had to print the no of valid ways distributions.

Approach: I thought of writing a recursive solution first and then optimizing it using memoization, but was only able to solve 5 test case. Thought of debugging it but the platform didn't allow to see print statements (tho after the test I got to know that you can see print statements via custom inputs). Gave approx. 45 mins to this problem but was not able to solve it completely

- 3) You had to find the count of unordered pair such that their sum is s and the total of set bits in both the number is $\leq k$

Approach: As the constraint was small enough for $O(N^2)$ to work but I solved it in $O(N)$ as it was quite similar to the famous 2 sum problem. So used a map to store numbers and if the complement ($s-a[i]$) is present in the map checked for the set bit condition.

- 4) The question didn't specify the no of edges so didn't even look at this question as I had no practice on how to take input of this kind. However, as the question only needed to print possible or not possible and each test case having only one result so just printing **possible** was enough to get 6/8 testcase right. And further as one of the wrong testcase was the sample one so hard coded it and got 7/8 right. Well, they did mention that code quality was being considered but taking into consideration that no one was able to solve this optimally and almost everyone printed possible, this question may as well have not been considered for the selection.

I solved 2 problems completely and 2 partially which was enough for being selected for the next round.

2. **[Technical Round 1]** [30 students appeared]

All the 30 students were divided into 3 panels of 10 each. I was part of the 1st panel.

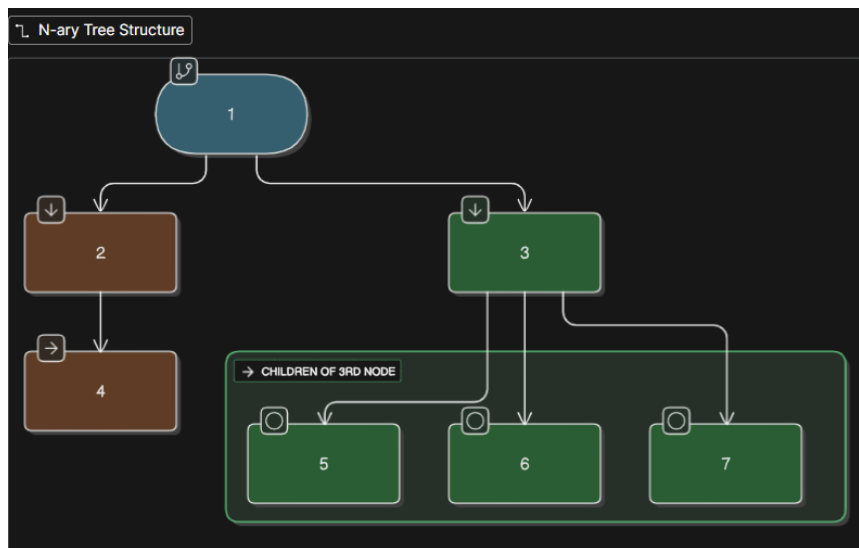
The interviewer introduced himself and ask for mine as well. I kept my intro short as I wanted more time for the 2 problems.

Question 1: Find the peak element in a 2d matrix. Where peak element is defined as the element which is greater than its 4 adjacent neighbors.

Before starting the question, I made to fully understand the problem by clarified the constraints, is it 4-directional or 8-directional adjacency. I started with the brute force solution which was to iterate and see if the condition is met.

Then I showed him how monotonicity is maintained in a 1d array for finding the peak element and how we can use binary search to implement it. Thought of flattening the 2d array and applying binary search on it. However soon found out that it does not work. With the help of some hints from the interviewer, found the optimal solution which involved binary search with the search space as the range of rows/cols. He then asked me to code the solution of paper.

Question 2: You were given a tree and q queries. Each query had a start node, an end node and a destination. The task was to find the node which is at a minimum distance from destination while also being on the path from start to end node.



Ex: Consider the n-ary tree
And the query [2,5,7].

The path from 2 to 5 would be 2->1->3->5 and out of all the node the node 3 is at minimum distance from 7 so the answer is 3.

It started by saying that we can precompute the distance between all pair of nodes (MSSP algorithm) by using Floyd-Warshall algorithm and then for each query find the path using dfs/bfs while also maintaining a parent array to track the path. And using the parent array to trace the path we see which node has the minimum distance from destination.

However, as the distance matrix takes $O(N^3)$ time to compute, it wasn't optimal so the interviewer asked me look the answer in terms of LCA (Lowest common ancestor). I tried to find some observations from the testcase and found that the answer can be one of the LCA (start, end), LCA (start, destination) and LCA (end, destination). Once these 3 nodes are found we can calculate the minimum distance from the destination. However, it was not optimal and time was up so left it there.

The interviewer then asked for any questions for him. I had researched about it and asked him.

I was not able to solve both the question however don't let it demotivate you. The interviewer is more interested on how you approach the question so be open to him and read out your thoughts out loud and ask him for any hints if needed.

To save some time, from each panel 3 were fast-tracked to give the HR round before 2nd tech round.

3. **[Technical Round 2]** [14 students appeared]

My 2nd tech round happened after my HR round. And similar to 1st tech round we were asked to solve 2 questions. I was not asked for my intro this time.

Problem 1: Given a matrix of size $m*n$, each element had a heat value which I gained if I landed on it, I had to return the minimum heat with which I could reach bottom right corner from top left corner under the following constraints:

1. I could move left, right, bottom or top.

2. I could not retrace my path at any point.
3. I couldn't move the same direction more than 3 times.

I initially approached the problem using BFS, but since we're trying to minimize a cumulative cost (heat), I pointed out that Dijkstra's algorithm would be more suitable here.

I took 5 states in my heap as heat, x, y, prevDirection, countOfPrevDirection and implemented the standard Dijkstra with the constraint on the paper while considering the code quality as well. I was asked a follow up that what if I had to move k steps before changing direction. I handled that as well

Problem 2: [The exact problem from leetcode](#)

I started with a greedy solution which as expected didn't work on a few cases. Then I thought about dp to which the interviewer said I was on the right track. I was able to find the state which is quite similar to 0/1 Knapsack but the interviewer was not satisfied. I then dry runned the test cases given by him and my solution worked. He was satisfied by my answer till the end of interview I then asked him what would the optimal solution be. He opened the leetcode question itself and showed the editorial. The editorial had the same solution as mine. He then shook hands with me and said "Well Done".

4. **[Hiring Manager Round]** [11 students appeared]

PhonePe's HR round isn't a typical HR round; it revolves more around the resume so be have good projects on it and (internship if any).

The interviewer was the engineer manager at PhonePe. He started by asking me to walkthrough my projects. Since two of them were hackathon-winning projects, I had a clear strategy on how to present them: I first explained the problem or need, then walked him through how I approached and solved it, which helped build curiosity around the tech stack I had used. He asked me about jwt and RSA algorithm. I was not sure about the exact implementation of it but gave a rough idea to him. In one of the projects, I had worked on the **entire software development lifecycle**, and I made sure to highlight that experience specifically.

I also made sure to explain him about a devOps experiment that I had done which wasn't mentioned on my resume. He was quite impressed by this.

He then asked me about my favorite app and its most used feature and I answered with reddit. He went forward to ask me to implement the post feature from the DB selection to the schema and optimizations (Pagination, caching, etc.).

He then followed with typical HR question. I gave honest and diplomatic answers to him using real life examples.

I was one of the first to have concluded with all the rounds so had to wait 4 hours for the results to be announced

7 students received the offer.

Useful Tips:

1. DSA is essential, you should be able to solve leetcode hard in a time constraint environment with some hints. The more questions you solve the better you will get at it.
2. Have good projects on your resume which you can explain thoroughly.
3. Have mock interviews with your friend so as to know where you can improve

Resources:

1. **DSA:** Follow [Strivers sheet](#) for the standard problem. You can also consider solving [CSES](#) problem sheet (or [TLE](#) sheet for updated level wise problems) and [USACO](#) for getting better at CP.
2. Here's my [resume](#)

For any further help feel free to Reach out to me - [LinkedIn](#)